Hashiko

Distributed File Integrity Monitoring System
with Temporal Correlation

Table of Contents

Document Control

Document Information

	Information
Document Owner	Jackie Ma
Document Owner ID	A00889988
Issue Date	2025-09-19
Last Saved Date	2025-12-02
File Name	comp8800_jackie_ma_proposal.pdf

Document History

Version	Issue Date	Changes
1.0	2025-09-19	Initiation for Proposal Draft 1
2.0	2025-10-26	Add List of Abbreviations and Scope and Depth, update all sections with provided feedback and add references for Proposal Draft 2
3.0	2025-11-09	Update sections with provided feedback, rewrite backend to Flask and add composite indexes for Milestone 1.
4.0	2025-12-02	Update Methodology and Design, Schedule and Milestones section

List of Abbreviations

AIDE - Advanced Intrusion Detection Environment
API - Application Programming Interface
BCIT - British Columbia Institute of Technology
BRIN - Block Range Index (PostgreSQL index type)
CI/CD - Continuous Integration/Continuous Deployment
CRUD - Create, Read, Update, Delete
CSV - Comma-Separated Values
ERD - Entity Relationship Diagram
FIM - File Integrity Monitoring
HIDS - Host-based Intrusion Detection System
HTML - HyperText Markup Language
HTTP - HyperText Transfer Protocol
HTTPS - HyperText Transfer Protocol Secure
ID - Identifier
IDS - Intrusion Detection System
IP - Internet Protocol
IT - Information Technology
JSON - JavaScript Object Notation
JSONB - JSON Binary (PostgreSQL data type)
JWT - JSON Web Token
mTLS - Mutual Transport Layer Security
NATS - Neural Autonomic Transport System (message queue)
NTP - Network Time Protocol
OS - Operating System
OSSEC - Open Source HIDS Security
PDF - Portable Document Format
REST - Representational State Transfer
SHA-256 - Secure Hash Algorithm 256-bit
SIEM - Security Information and Event Management
SQL - Structured Query Language
TLS - Transport Layer Security
UI - User Interface
URL - Uniform Resource Locator
UTC - Coordinated Universal Time

Abstract

Hashiko is a distributed monitoring system that tracks file integrity and user activity. It detects threats by linking events through time and sequence, such as login, privilege change and file edit. Agents collect data with timestamps and send it to a central server. This agent-server design enables centralized monitoring across distributed systems. The server builds event chains and generates alerts. A web interface shows real-time alerts, timelines and rules. The system focuses on accuracy, secure transfer and scalability. Testing uses synthetic data, log replays and simulations to validate components. Scope includes agents, server, storage and interface. Out of scope are malware analysis and compliance reporting. Hashiko improves detection by closing temporal blind spots.

Student Background

I am a fourth-year student in the Bachelor of Applied Computer Science program at BCIT, specializing in Network Security Applications Development. My academic background includes a Computer Systems Technology Diploma from BCIT, specializing in Web and Mobile development. My technical foundation includes proficiency in multiple programming languages: Python, Java, JavaScript, C#, SQL, Swift and Kotlin. I also work with web technologies including HTML and CSS. Frameworks and platform experiences include .NET, Node.js, Express.js, Flask and Laravel. Database experience spans PostgreSQL, MySQL, MariaDB, SQLite and Firebase.

Through previous coursework and personal projects, I have developed several full-stack, backend and database-driven applications. For example, PromptVisioQuiz fetches data from external APIs (CBC RSS feed) and processes it through a multi-service architecture with Node.js/Express and Python/Flask microservices, using JWT authentication and PostgreSQL for data persistence. MealPlanIQ demonstrates my ability to build complex data-driven applications with TypeScript/Angular frontend and Python/Flask backend, managing relational data with PostgreSQL and deploying to cloud infrastructure (GoogleCloud Platform). These projects gave me practical experience in distributed system architecture, RESTful API design, secure authentication, database management andcross-service communication—skills directly applicable to Hashiko's agent-server architecture, event processing pipeline and temporal data storage requirements.

To prepare for this project, I researched file integrity monitoring techniques and distributed security architectures. I read Kaczmarek and Wrobel's analysis of Modern Approaches to File System Integrity Checking approaches to understand current detection methods and their limitations. Their review of user-space and kernel-space integrity checkers helped me understand the trade-offs between periodic checking and real-time monitoring approaches. I also read Abdullah et al.'s work on File Integrity Monitor Scheduling Based on File Security Level Classification scheduling. Their research on combining off-line and on-line monitoring informed my approach to resource prioritization. Beyond these foundational papers, I have explored existing distributed monitoring tools such as OSSEC and Wazuh to understand

agent-server communication patterns. I am currently investigating time-series databases for efficient event storage and message queue systems for reliable data transmission between agents and the central server.

Before studying computer science, I studied architecture and briefly worked in the industry. This background has strengthened my analytical and problem-solving abilities. Examples of my work, such as PromptVisioQuiz and MeanPlanIQ, can be found at www.jackiema.ca. After graduation, I plan to work in software development or IT.

Project Summary

This project is motivated by the growing need for proactive and context-aware threat detection in modern distributed systems. As attacks increasingly exploit timing gaps between independent events, identifying relationships across time has become essential for effective defence. Current security monitoring tools create dangerous blind spots. Traditional intrusion detection systems and file integrity monitors detect individual events but miss the bigger picture. They cannot connect actions that occur across time. A suspicious login, followed by privilege escalation, then a file modification might each seem normal alone. Together, they reveal an attack in progress. This temporal gap allows sophisticated threats to slip through undetected. Organizations need a way to see patterns that only emerge when events are linked by timing and sequence.

Hashiko is a distributed monitoring system that tracks both file integrity and user activity. The project implements a scalable, time-aware monitoring framework that correlates events to expose multi-step attack sequences invisible to traditional tools. It goes beyond typical IDS tools by detecting threats through timing and sequence patterns. The focus is on temporal relationships. For example, sensitive files being modified right after an unusual login or privilege changes.

Endpoint agents collect file hashes, login sessions and process activity. All data is sent with timestamps to a central server. The server builds time-windowed event chains and flags patterns that isolated event-based systems often overlook. Alerts trigger when suspicious sequences match predefined or custom temporal threat patterns. A web interface provides clear visibility, easy alert tracking and quick access to detailed event data. Thus, the administrator has a centralized and readable view of ongoing risks.

The goal is to enable temporal threat detection that conventional tools cannot provide. By correlating events across endpoints within specific time windows, the system identifies multi-step attacks and insider threats. Expected outcomes include earlier breach detection as attack patterns emerge in real time. False positives will decrease because sequence context separates normal workflows from malicious activity. Security teams will investigate faster using timeline views that reveal complete attack chains. The system transforms isolated security events into connected intelligence. This provides organizations with actionable insights and significantly improved protection against time-based attack patterns.

Problem Statement and Background

Organizations face increasing risks from both external attackers and insider threats. Traditional tools such as intrusion detection systems or file integrity monitors can detect single events. Tools like Tripwire and OSSEC use hash comparison to detect file changes [1,2]. AIDE, Samhain and Wazuh also perform file integrity monitoring. AIDE and Samhain use scheduled hash checks. Wazuh extends OSSEC with central logging and rule-based alerts. These tools detect isolated events. They do not analyse time or event order. Studies by Abdullah et al. and Kaczmarek and Wrobel confirm that periodic and user-space methods delay detection [1,2]. Their periodic checking creates detection delays and higher inspection frequency improves detection but degrades performance. User-space integrity checkers can only verify if files were modified between checking operations [1]. During this period, attackers can replace system files with backdoors or trojans that may be executed by authorized users [1]. However, they rarely correlate activity over time. Thus, it leaves blind spots where subtle yet dangerous patterns go unnoticed. For example, an unusual login, followed by privilege escalation and then a file change, may seem harmless in isolation but is critical when viewed as a sequence.

The core weakness is the lack of temporal awareness in existing monitoring systems. Timing order and relationships between events are as important as the events themselves. Traditional FIM tools face a fundamental trade-off. High-frequency inspection maintains effectiveness but hurts performance. Low-frequency inspection reduces overhead but allows threats to persist undetected. Manual policy configuration becomes impractical in distributed environments [2]. Without this context, significant risks bypass defences. SIEM tools like Splunk and Elastic Security try to improve visibility. They collect and correlate logs but use static rules. They cannot detect multi-step attacks spread over time. Temporal context is still missing. Visibility is further reduced by scattered logs and rigid dashboards that slow incident response. A centralized system that aggregates events from multiple endpoints, applies timestamps and links them together can close this detection gap. Centralized monitoring is essential for security. Attackers with administrator privileges can compromise local security tools. Remote monitoring from a privileged server provides protection even when the monitored system is compromised [2]. A web interface adds readability by simplifying alert tracking and management.

Hashiko closes this gap. It uses agents that timestamp events and send them to a central server. The server links actions by time and sequence. This method combines the reach of SIEM tools with the detail of host monitors. It delivers real-time, context-aware detection instead of isolated alerts.

Inventiveness

What sets Hashiko apart is its focus on temporal event correlation. Most security tools detect single anomalies, such as a file change or a suspicious login. The project instead looks at the timing and sequence of events across endpoints. It flags patterns that become suspicious only when actions occur together, within specific time windows. Existing studies focus on improving

detection accuracy through frequency tuning and signature updates [1,2]. Few examine temporal relationships between heterogeneous events such as logins and file access. This correlation gap remains underexplored in host-based intrusion detection research.

Another inventive aspect is the centralized, distributed design. Endpoint agents collect activity data such as file hashes, logins and processes. This follows established HIDS deployment patterns. However, it extends beyond traditional file checking. It incorporates authentication and privilege events into correlation. They then attach precise timestamps before sending it to the central server. Instead of producing isolated alerts, the system creates linked event chains. A web interface presents these chains in a clear way, giving teams both context and visibility. This blend of distributed data gathering with temporal threat analysis is rarely seen in current monitoring solutions. While some commercial SIEM systems such as Splunk Enterprise Security and IBM QRadar provide event correlation, they rely on pre-defined static rules and lack real-time temporal chaining at the host level. Academic systems like I3FS and SOFFIC achieve real-time monitoring but require kernel-level modifications and limited portability. Hashiko's user-space approach provides cross-platform deployment without kernel dependencies, combining practicality with deeper time-aware analysis.

Traditional FIM solutions require manual policy configuration. Administrators must specify which files to monitor and how often. This becomes impractical in large-scale environments. Hashiko automates threat detection through temporal correlation rules. Unlike systems that monitor only files or logs, Hashiko correlates multiple event types. Traditional file integrity checkers like Tripwire, AIDE and AFICK work similarly—they all perform periodic hash comparisons. Hashiko differs by correlating file modifications with temporal user activity rather than treating each file change as an isolated event. This reveals attack sequences that individual tools would miss.

Complexity

The project is complex as the system must collect data from many endpoints and link events in real time with precise timestamps. This makes it significantly more advanced than simple single-event detection. The system integrates distributed agents, secure real-time communication, temporal correlation algorithms and scalable event storage. The most difficult aspect is linking events by time and sequence, which is far more challenging than detecting isolated anomalies. Success requires skills in event processing, threat modelling and testing of correlation models. This project requires independent design decisions that go beyond coursework. Trade-offs between detection accuracy and system performance must be experimentally evaluated.

The correlation engine requires designing efficient pattern matching algorithms that operate on time-windowed event streams. This exceeds traditional FIM approaches. Tools like Tripwire perform simple hash comparisons, while temporal correlation requires stateful pattern tracking across event streams. Traditional integrity checkers simply compare current hash values with baseline patterns stored in a secure database. It requires maintaining partial pattern states across distributed agents, handling out-of-order events and efficient time-series queries.

Conventional integrity monitoring tools do not address these challenges. Implementing time-windowed correlation requires original algorithm design. No off-the-shelf solution exists for multi-event temporal pattern matching in FIM contexts. The system integrates distributed architecture, real-time event processing and security monitoring into a cohesive platform. This combination of distributed data collection, temporal analysis and secure communication represents Bachelor-level integration. The project demands trade-off analysis between memory overhead and pattern detection speed.

Technical Challenges

Distributed agents must be designed to securely collect and transmit data. Authentication mechanisms are needed to prevent agent spoofing. File integrity databases must be secured because if an attacker modifies the database, the entire security system becomes unreliable. Most integrity checkers store databases on read-only devices or use cryptographic functions to protect them. Secure agent-server communication is critical. This is especially true when agents operate on potentially compromised systems. Event data must use encrypted channels to prevent interception. Network interruptions must be handled without losing events. Ensuring exactly-once delivery requires handling network failures and retries without creating duplicates. Message acknowledgments must be tracked with unique IDs. Out-of-order acknowledgments will cause temporary failures, requiring retry logic with exponential backoff. Learning message queue systems like NATS is necessary to implement these delivery guarantees. Clock synchronization across distributed endpoints is challenging due to network latency and drift. NTP synchronization will be used, but correlation logic must handle events arriving out-of-order by buffering and reordering within configurable time windows. This timing challenge is more complex than traditional FIM. Off-line tools like AIDE schedule periodic checks at fixed intervals. Real-time correlation requires precise timestamp synchronization. This ensures correct event ordering in multi-step attack sequences. Learning NTP client libraries and designing clock drift detection mechanisms is necessary.

A central server will be required to store, process and correlate large event streams in real time. It must handle event chaining, sequence detection and scaling across multiple hosts. The correlation engine must maintain partial pattern states across distributed events. For example, detecting login from an unusual location, followed by privilege escalation within one minute, then sensitive file access within two minutes without overwhelming the server. Storage must support fast queries across multiple dimensions. Traditional FIM tools store baseline databases that are updated periodically, requiring significant maintenance overhead. Hashiko's model differs fundamentally by supporting high-velocity event ingestion and enabling complex temporal queries across multiple dimensions. These include time range, endpoint, user and file path while ingesting thousands of events per second. This requires composite indexes and time-series partitioning. Learning PostgreSQL's declarative partitioning and BRIN indexes is required for time-series optimization.

The web interface presents challenges in real-time state management and rule visualization. Learning React with WebSocket integration is necessary for live alert updates. The rule builder

must translate visual drag-and-drop logic into temporal predicates, requiring understanding of abstract syntax trees and query generation. SIEM correlation techniques from industry research must be studied and adapted to the FIM context. Multiple pattern matching approaches will be prototyped and benchmarked to determine optimal performance trade-offs between memory overhead and detection speed.

Methodology and Design

The project follows an agent–collector–correlator design with a central server and web interface. This architecture separates data collection from analysis. This pattern appears in multi-platform HIDS deployments. Centralized management enables efficient monitoring of multiple systems. It addresses scalability limitations of standalone FIM tools. Standalone file integrity checkers like Tripwire were originally designed to work independently before becoming integrated into comprehensive intrusion detection systems. Hashiko's architecture follows this evolution by integrating file integrity monitoring with authentication and privilege monitoring from the start. Agents compute file hashes, monitor file paths, record logins and send timestamped events. The server ingests, validates and stores events. Then the server applies sequence rules to construct time-ordered chains that expose multi-step threats. The web interface delivers real-time alerts, timelines and rule management for rapid triage and investigation.

System Architecture

Figure 1 illustrates Hashiko's architecture. Endpoints run Linux agents that monitor activities, logins and processes. Agents synchronize time via NTP and transmit timestamped events with hashes to the Ingest API via HTTP POST. The Normalizer validates data and applies UTC conversion to ensure consistent timestamps across distributed endpoints. Normalized events stream to the Correlation Engine, which applies temporal rules and maintains baselines to detect multi-step attack sequences. When patterns match, the Alert Service generates alerts with severity and ticket metadata. The web interface provides real-time dashboards via WebSocket, a REST API for programmatic access and a Rule Manager for defining temporal patterns. Storage uses an append-only Event Store for immutability, versioned Config/Rules for change tracking and composite indexes on time, entity and path for efficient queries. External integrations enable alert forwarding to ticketing systems and annotations for incident tracking.

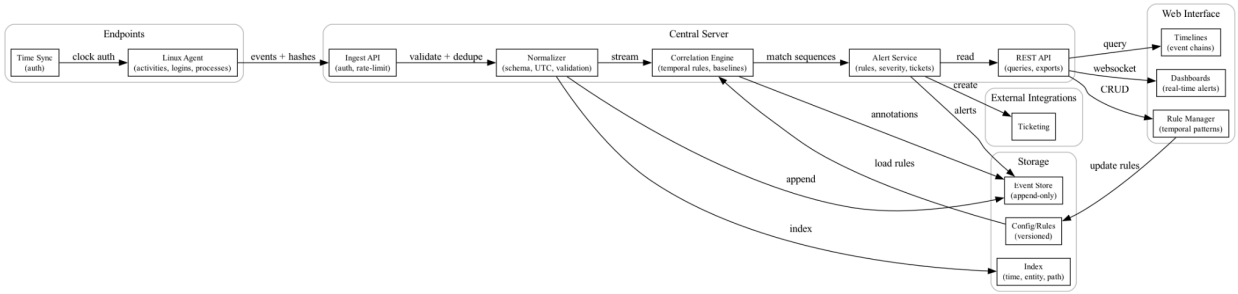

Figure 1 - UML Deployment Diagram

Agent Architecture (Current Implementation)

The agent uses SHA-256 hashing via crypto/sha256. SHA-256 is a cryptographic hash function that produces fixed-size output regardless of input length. This ensures that even a single bit change in a file generates a different hash value. The hashDirectory function walks directories with filepath.Walk and hashes file paths and contents. Five directories are monitored: /boot, /usr/bin, /usr/sbin, /etc and /root. These directories contain common attack targets [2]. System files control core operating system functions. Malicious modifications can disrupt services. They can enable attacks on other systems [2]. Excluding volatile paths like cache reduces false positives. It also reduces performance overhead. Volatile paths like .cache, .mozilla, tmp and session files are filtered. JSON output includes timestamp and directory hashes. Transmission via HTTP POST to /api/store.

Server Architecture (Current Implementation)
Built with Flask and PostgreSQL. Loads environment variables and creates a hash_records table with id, timestamp and five hash columns indexed by id descending. The handleStore endpoint decodes JSON, normalizes client IP and inserts records with parameterized queries. The handleView endpoint returns all records as JSON ordered newest first. Serves static files on port 8800.

Development
Development will follow an incremental approach. Milestone 1 implements agent data collection, basic server storage, SHA-256 hashing, HTTPS and composite indexes. Milestone 2 adds message queuing and retry logic. Milestone 3 builds the correlation engine and adds NTP. Milestone 4 develops the web interface. Milestone 5 includes deployment, tests and documentation. Each milestone includes unit, integration and system testing before proceeding. Version control with Git enables rollback if issues arise.

Scalability
This modular design enables horizontal scaling. Multiple server instances can process events concurrently using a shared PostgreSQL database. Weekly partitioning allows old partitions to be archived without impacting active queries. Composite indexes on (timestamp, agent_id, event_type) enable query parallelization across partitions.

Interface (Current Implementation)
The HTML table shows ID, timestamp and five hash columns. JavaScript fetches from /api/view and populates table rows.

Database Schema (Current Implementation)
The hash_records table has auto-incrementing id, text timestamp and five hash columns for SHA-256 strings. Index on id descending optimizes queries. Limitations include no agent tracking, no event types and text timestamps that prevent range queries. The planned migration addresses these shortcomings. Typed events with proper timestamps enable temporal correlation. This distinguishes Hashiko from traditional integrity monitors.

Planned Database Migration

Four tables will replace hash_records. Events table stores event_id, agent_id, timestamp (proper timestamp type), event_type (login, privilege_escalation, file_modification, process_execution), payload (JSONB with event details) and sequence_number. Agents table tracks agent_id, hostname, os_type, last_seen and certificate_fingerprint. Rules table defines rule_id, name, pattern (JSONB), time_window_seconds, severity and enabled flag. Alerts table contains alert_id, rule_id, timestamp, matched_events (JSONB array) and status. Composite indexes on timestamp, agent_id and event_type enable fast queries. Weekly partitioning manages growth. Migration converts hash_records to file_modification events.

This four-table design separates concerns effectively. It improves upon single-table FIM implementations. The separation enables dynamic rule management. Traditional FIM tools require manual configuration. The Rules table implements temporal correlation logic. Existing tools lack this capability.

Planned Enhancements
Activity monitoring for logins (/var/log/auth.log), privilege escalation (sudo) and process execution. Extended JSON schema with event_type, agent_id, ntp_offset_ms and sequence_number. NTP syncs every 300 seconds with drift flagging over 1000ms. Correlation engine with in-memory hash table keyed by (agent_id, user) evaluates rules, tracks partial matches, generates alerts on pattern completion and expires old matches. Four-table schema: events, agents, rules and alerts. Composite indexes on (timestamp, agent_id, event_type) with weekly partitioning and 90-day retention.

Maintainability
The four-table schema improves maintainability over single-table FIM. Rules can be modified without schema changes. New event types require only payload JSONB updates. Agent registration is independent of event storage. This separation enables independent component updates and testing.

Test Plan

The project will be tested in a staged lab. Synthetic generators, safe log replays and controlled adversarial simulations will be used. This addresses a key limitation in FIM research. Evaluating detection effectiveness is difficult without compromising production systems. File integrity checkers are typically tested either through periodic manual execution by administrators or through scheduled runs using services like Cron. Hashiko's testing approach simulates attack scenarios rather than waiting for periodic checks. Controlled simulations enable safe validation. These tests will validate agents, transfer, normalization, correlation, storage, APIs and the web interface. Verification uses clear checkpoints for accuracy, throughput, detection latency, durability and UI responsiveness. A release moves forward only when all checkpoints are passed.

Unit Testing

Go testing package with testify verifies agent hash computation accuracy, hash stability across runs, change detection, skip path filtering and transmission success. Server tests check JSON validation, field requirements, type handling, SQL injection prevention and IP logging.

Integration Testing
End-to-end test runs server and agent in separate processes. Agent transmits data. Query verifies record appears within 5 seconds with correct fields. Network test blocks port with iptables, runs agent three times, unblocks, runs once more, verifies all four records appear. Future correlation test injects timed events and verifies alert generation.

Performance Testing
The load test runs 10 agents every 30 seconds for 10 minutes. Measures response time (target p95 under 200ms), throughput (over 20/second) and memory (under 100MB). Stress test scales to 100 agents finding breaking point. Query test measures response time with 1000 to 100,000 records (target under 500ms).

System Testing
Insider threat simulation includes login → sudo → /etc/passwd edit → data copy. The /etc/passwd file controls user authentication. Modification by an administrator appears legitimate in isolation. But when preceded by unusual authentication and followed by data exfiltration, the sequence indicates insider threat. This test validates Hashiko's temporal window linking. Compromised account simulation includes foreign IP login → failed sudo attempts → successful sudo → useradd. Mass modification simulation includes suspicious processes → 20 rapid file changes. Current implementation detects hash changes. Future generates severity-based alerts with timelines.

These scenarios simulate real-world attack patterns. Adversaries gain privileges and modify system files. Traditional FIM would detect the file modification in isolation. User-space integrity checkers can only detect that a file changed, not prevent the execution of compromised files [1]. They do not protect the file system in real-time [1]. Traditional FIM would not correlate file modifications with preceding authentication events and might treat them as legitimate administrator activity. Hashiko's temporal correlation provides context that helps distinguish legitimate administrative changes from malicious sequences and should flag the entire attack chain.

Test Environment
Main testing on desktop with Fedora 42, Python 3.14 and Go 1.24.8. Secondary testing on MacBook M1 Air for cross-platform validation. Future testing on school lab machines for multi-endpoint testing with Linux and Windows.

Test Data Management
Test data includes synthetic event logs with known attack patterns, baseline hash databases for integrity checking and timed event sequences for correlation validation. Controlled datasets enable reproducible testing and regression detection. Test data versioning ensures consistent results across test runs.

Success Metrics
Over 80% code coverage (current 0%), under 1% integration errors, zero false negatives on file integrity (current: /etc, /boot, /usr/bin, /usr/sbin, /root; future: temporal patterns), under 5% false positives, performance targets met, no critical/high bugs. Testing in 2-week sprints with regression. GitHub Actions automates testing on every commit. Unit tests run first (fail fast), followed by integration tests if units pass. Failed tests block merges, preventing broken code from entering the main branch. Weekly system/performance tests. Final 48-hour soak test with 100 agents at 60-second intervals validates stability and resource usage. When tests fail, root cause analysis identifies whether issues stem from code defects, test design flaws or environmental factors. Critical failures halt development until resolved. Non-critical issues are tracked as technical debt and addressed in future sprints.

Expected Outcomes
- The system detects unauthorized file modifications.
- Alerts are generated when rule conditions are matched.
- No events are lost during network interruptions.
- All significant test cases result in measurable logs and clear status (pass/fail).
- False positives are minimized during normal (benign) usage.

Sample Test Cases
- Detect modified critical file:
    - Edit /etc/passwd on a monitored agent.
    - Expect an alert and event log showing detection of the change.
- Simulate privilege escalation chain:
    - Login as a user, run sudo for escalation, then change /etc/shadow.
    - Expect the correlation engine to link these steps and generate one composite alert.
- Replay normal activity log:
    - Feed benign logins and normal file writes.
    - Expect zero alerts; confirms low false positive rate.
- Network failure simulation:
    - Disconnect agent network during events, then reconnect.
    - Expect events are queued, delivered after reconnect and the system state matches the test input (no lost/duplicate events).
- Out-of-order event handling:
    - Inject events with timestamps out of sequence.
    - Expect the system to correctly correlate and order alerts.

Scope and Depth

Scope includes agents, a central server, storage and a web interface. Features cover file integrity with baselines and hashes, login and process monitoring, normalised timestamped events, secure transfer, alerts, search and rule management. Hashiko employs hash-based integrity checking similar to Tripwire and OSSEC. However, it extends beyond simple baseline comparison by incorporating hash verification into temporal event chains. A file modification becomes evidence in a larger attack pattern. It is not just an isolated alert. Deliverables are another OS agent, a central pipeline, indexed storage, web interface, documented APIs, starter rules and a test plan.

Out of scope are memory scanning, kernel exploit detection, malware analysis and compliance reports. These exclusions distinguish Hashiko from comprehensive HIDS platforms like OSSEC. OSSEC includes rootkit detection and compliance reporting. Hashiko's focused scope enables deeper analysis of temporal correlation. It does not attempt to replicate full HIDS functionality.

Depth comes from accurate time sync, sequence-based correlation, scalable ingest, durable storage with replay, explainable rules, timeline views, custom temporal patterns, baseline learning and full security for identity, encryption and audit. This depth differentiates Hashiko from traditional FIM. Tools like Samhain and AIDE provide centralized management and periodic checking but lack real-time correlation engines and temporal pattern matching. Some advanced systems like I3FS and SOFFIC operate at the kernel level to provide real-time file checking. However, these require kernel modifications and are platform-specific. Hashiko takes a different approach—using user-space agents with centralized correlation rather than kernel integration. Hashiko addresses the performance-versus-effectiveness trade-off. It focuses resources on correlation rather than continuous file hashing.

Schedule and Milestones

This project spans two academic terms. It covers both COMP 8800 and COMP 8900. The following tables show a detailed week-by-week schedule. The plan outlines all key project milestones. This includes research, implementation and testing.

COMP 8800 focuses on core features. The goal is a functional prototype. It also includes the finalized project proposal. COMP 8900 builds advanced features. This term focuses on hardening the system. It concludes with the final project submission and presentation. All deliverables align with course milestones.

COMP 8800		
Week	**Key Activities & Focus**	**Due Date / Milestone**
1	Formulate project idea. Draft sections: Student Background, Project Summary.	
2	Draft sections: Inventiveness, Complexity, Technical Challenges.	
3	Draft Problem Statement. Revise Project Summary. Compile sections 1-6.	Proposal Draft 1 Due
4	Build initial prototype: agent hashing, basic server endpoint for data ingest.	
5	Refine prototype. Draft Methodology and Test Plan sections based on findings.	
6	Complete and document prototype for submission. Draft Scope, Schedule and References.	Prototype Due
7	Revise all proposal sections, incorporating insights from the prototype.	Proposal Draft 2 Due
8	Address initial feedback on Draft 2. Begin Milestone 1: Implement composite index and rewrite backend to Flask.	
9	Continue development on Milestone 1 deliverables.	
10	Complete all coding and documentation for Milestone 1.	Milestone 1 Due
11	Review feedback from instructor and second reader. Plan proposal revisions.	
12	Revise proposal based on all feedback. Begin Milestone 2: Implement message queue and retry logic.	
13	Continue development on Milestone 2 deliverables.	
14	Continue Milestone 2. Address proposal feedback.	
15	Integrate all Milestone 2 components. Complete Final Proposal, demo and give a presentation.	Final Proposal & Milestone 2 Due

COMP 8900		
Week	**Key Activities & Focus**	**Due Date / Milestone**
1 - 3	Begin Milestone 3: Implement correlation engine.	
4 - 5	Refine correlation engine with stateful pattern matching.	Milestone 3 Complete
6 - 8	Begin Milestone 4: Develop the interactive web UI for rule management.	
9 - 10	Implement real-time dashboard updates using WebSockets.	Milestone 4 Complete
11 - 12	Begin Milestone 5: Conduct large-scale performance soak tests. Write user and developer documentation.	
13 - 14	Perform final validation testing. Prepare capstone presentation and demo script.	Milestone 5 Complete
15	Deliver final presentation. Submit all project artifacts.	Final Project Submission

References

[1] J. Kaczmarek and M. Wrobel, “Modern Approaches to File System Integrity Checking,” *2008 1st International Conference on Information Technology*, pp. 1–4, May 2008. doi:10.1109/inftech.2008.4621669

[2] Z. H. Abdullah, N. I. Udzir, R. Mahmod and K. Samsudin, “File Integrity Monitor Scheduling Based on File Security Level Classification,” *Communications in Computer and Information Science*, pp. 177–189, 2011. doi:10.1007/978-3-642-22191-0_16